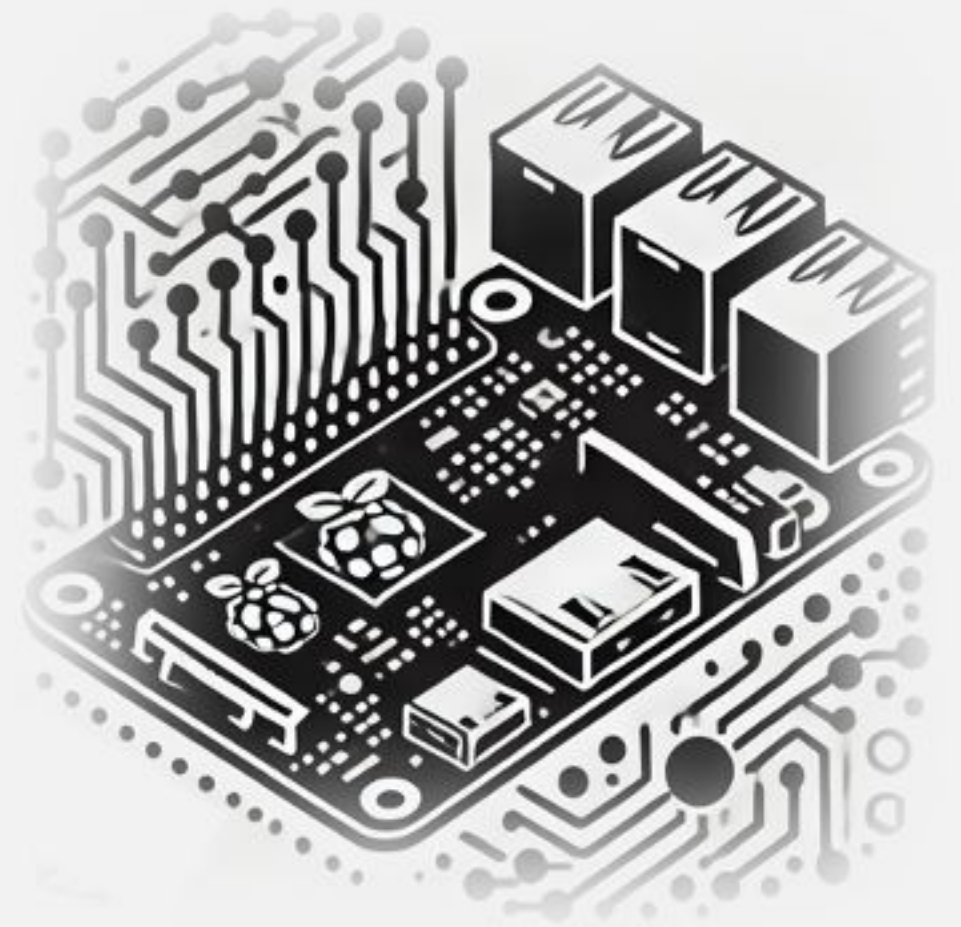


IESTI05 – Edge AI

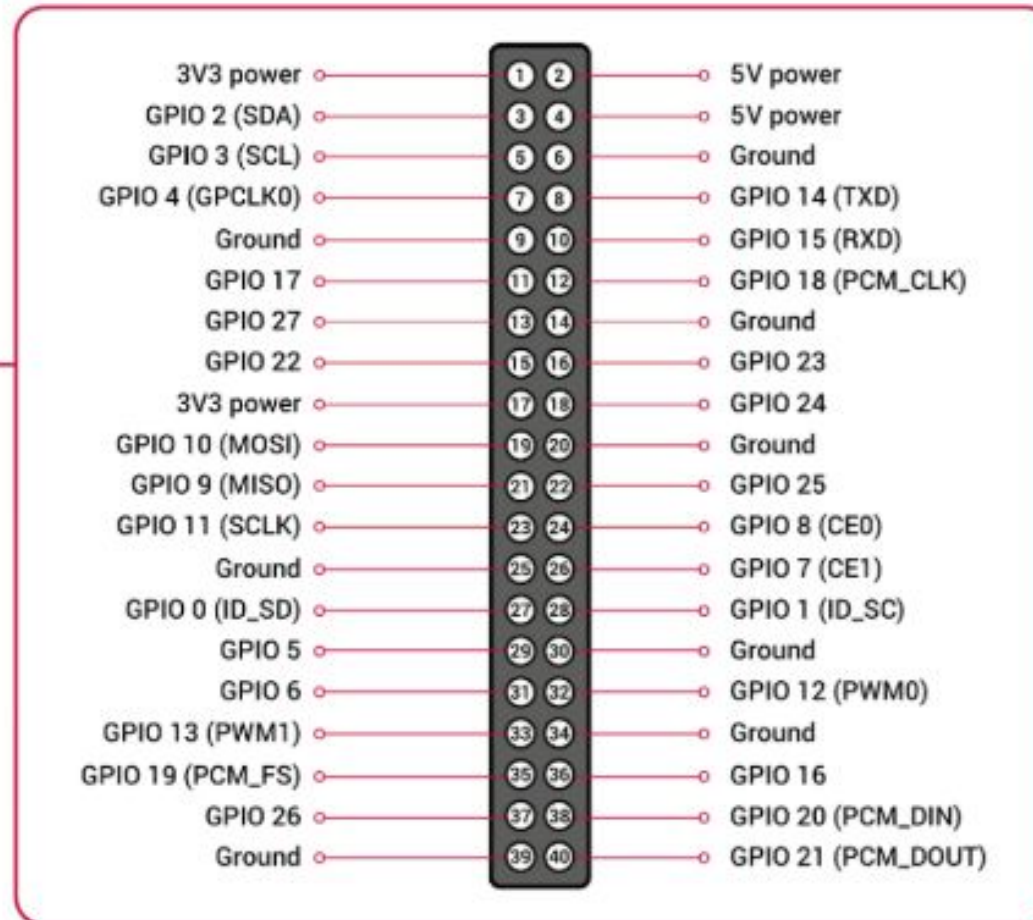
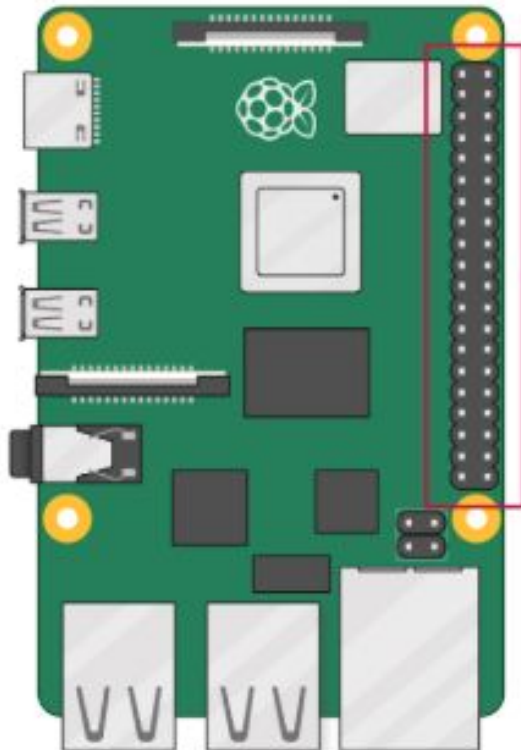
Machine Learning System Engineering

21. Physical Computing with the Raspberry Pi



General Purpose Input/Output (GPIO)

Pin Numbering Systems



There are two ways to reference GPIO pins:

- **BCM (Broadcom):** Uses the GPIO number (e.g., GPIO17, GPIO27)
- **BOARD:** Uses the physical pin number on the header (e.g., Pin 11, Pin 13)

GPIO 13 -> Pin 33

Main GPIO Libraries

1. RPi.GPIO – Does not work with the Raspi5
2. libgpio - Works with any Linux SBC
3. lgpio - Simple GPIO access (Python/C)
4. GPIO Zero – High-level Python interface
5. Adafruit Blinka* - Uses libgpio under the hood
6. smbus2 - I2C/SMBus protocol control

* Bridges CircuitPython libraries to Pi and supports dozens of sensor/device modules.

Safety First

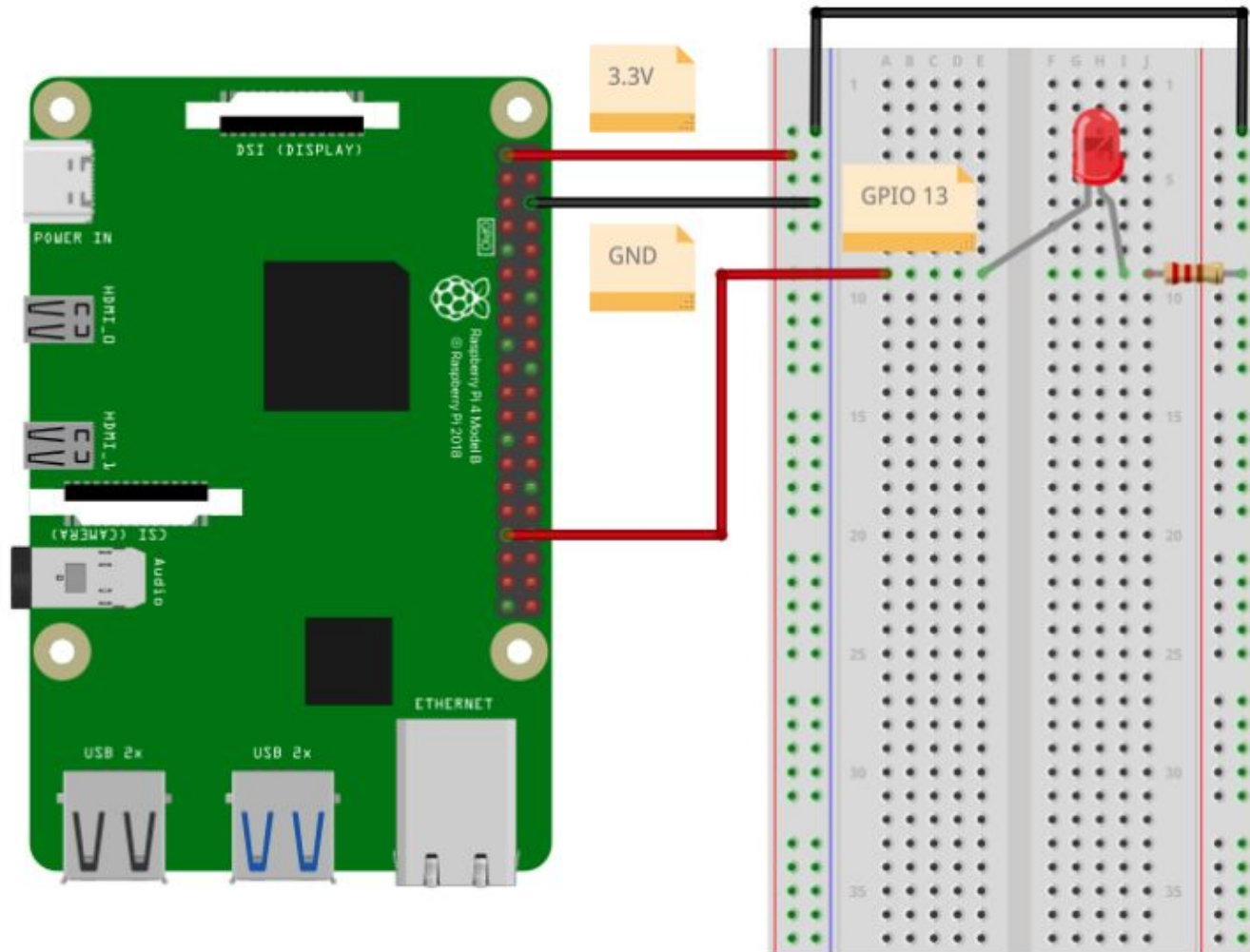
Before connecting any components, please read these important safety guidelines:

- **Never connect 5V directly to GPIO pins** - GPIO pins are 3.3V tolerant only
- Always use current-limiting **resistors with LEDs** - Without them, you risk damaging the LED or GPIO pin
- Double-check your connections before powering on
- Disconnect power when making circuit changes
- Respect polarity - LEDs, for example, have positive (long leg) and negative (short leg) sides

Actuators and Sensors

Installation & Tests

Hello World

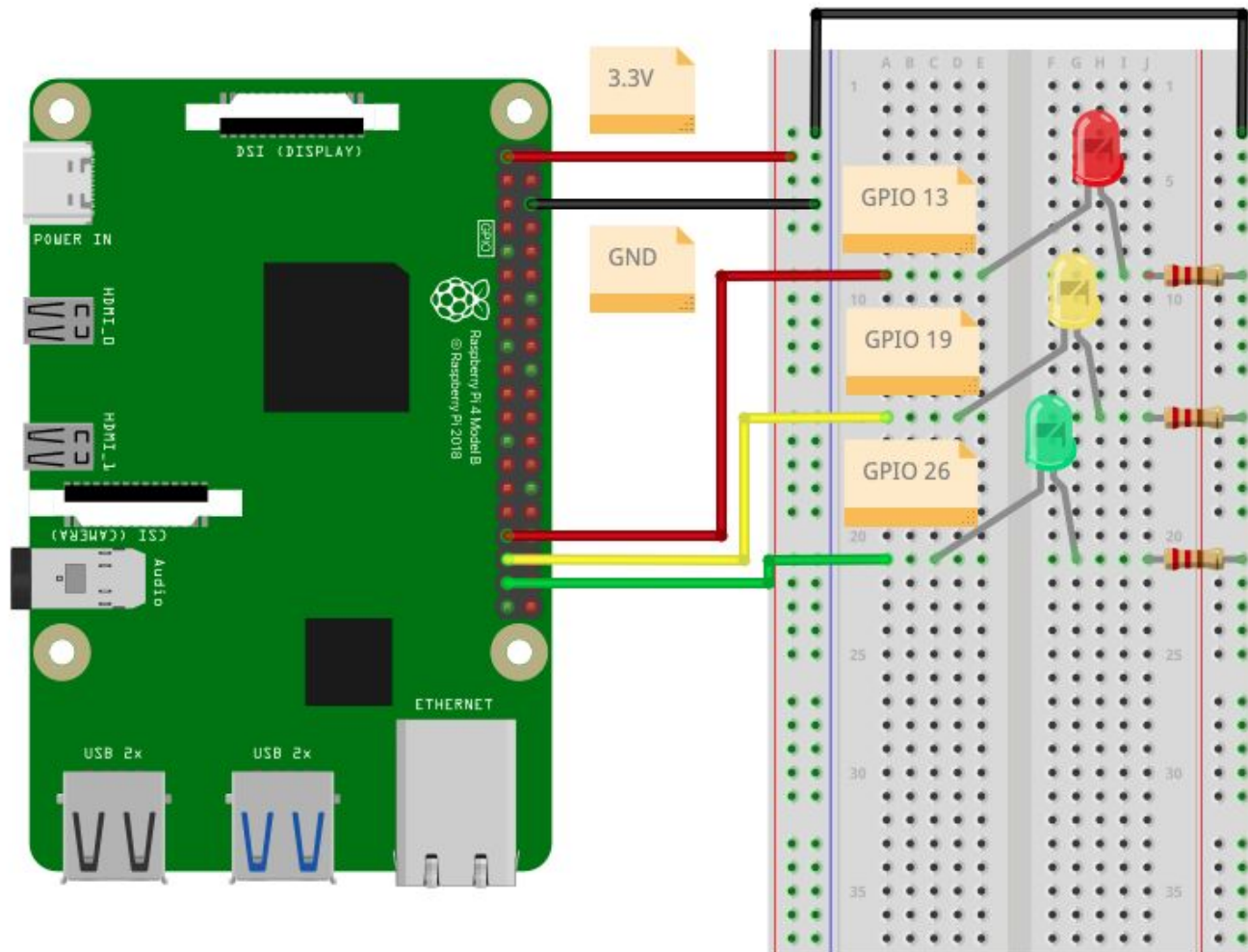


```
from gpiozero import LED
from time import sleep
led = LED(13)
while True:
    led.on()
    sleep(1)
    led.off()
    sleep(1)
```

- GPIO Zero comes pre-installed on Raspberry Pi OS!
- Note that the GPIO Zero Library uses Broadcom (BCM) pin numbering for GPIO pins

[The MagPi Essentials eBook: GPIO Zero](#)

The “Actuators”



```
from gpiozero import LED
```

```
ledRed = LED(13)
```

```
ledYlw = LED(19)
```

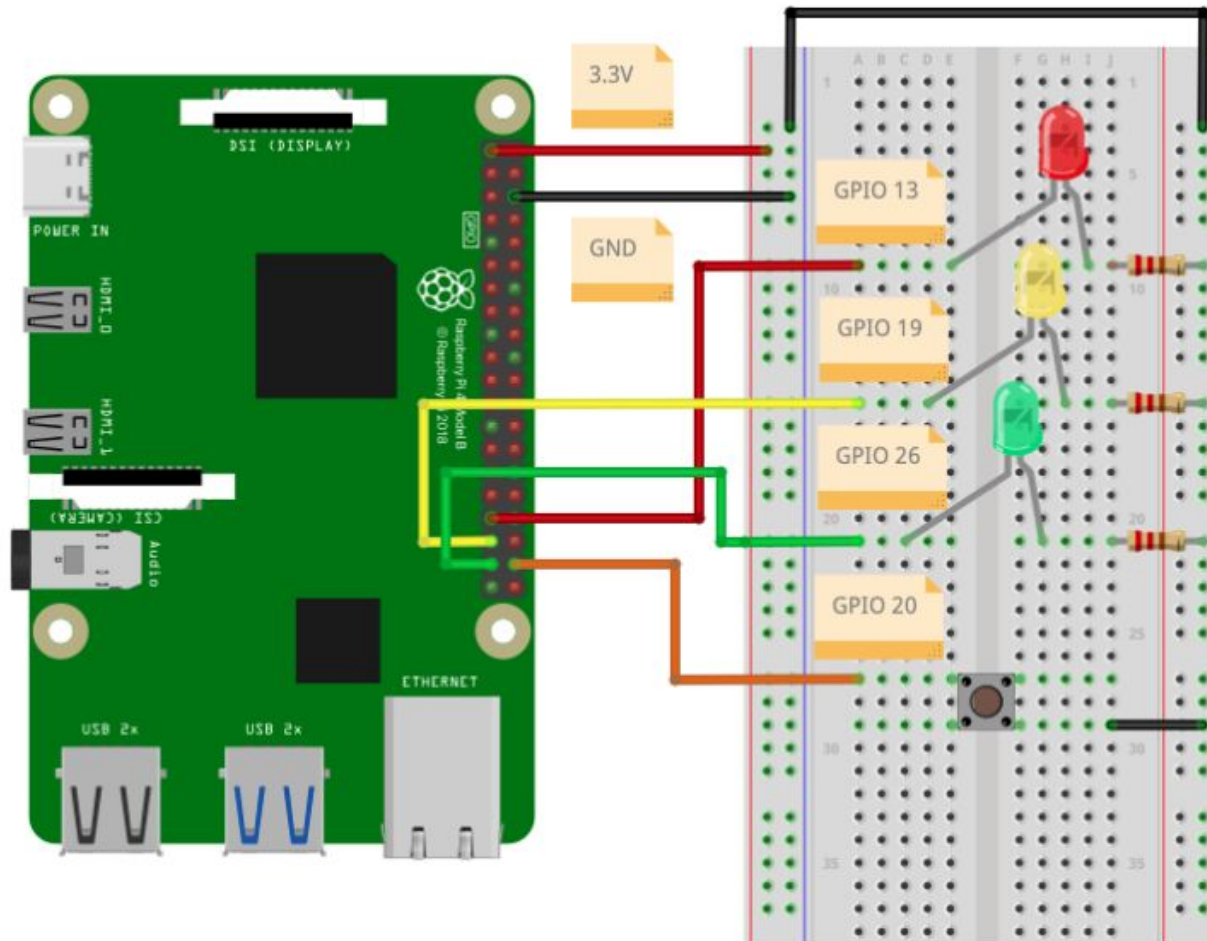
```
ledGrn = LED(26)
```

```
def controlLeds(r, y, g):  
    if (r): ledRed.on()  
    else:   ledRed.off()  
    if (y): ledYlw.on()  
    else:   ledYlw.off()  
    if (g): ledGrn.on()  
    else:   ledGrn.off()
```

```
controlLeds(1, 1, 1)
```

```
controlLeds(0, 0, 0)
```


Button (Digital Sensor)



```
from gpiozero import Button

button = Button(20)

if button.is_pressed:
    print("Button pressed")
else:
    print("Button not pressed")
```

Adafruit Circuit Python **Library**

Enable Interfaces

```
sudo raspi-config nonint do_i2c 0  
sudo raspi-config nonint do_spi 0  
sudo raspi-config nonint do_serial_hw 0  
sudo raspi-config nonint disable_raspi_config_at_boot 0
```

Install Dependencies

```
sudo apt-get install -y python3-libgpiod i2c-tools libgpiod-dev
```

Install Blinka (inside env)

```
source ~/ollama/bin/activate  
pip install adafruit-blinka
```

Test Blinka Library

```
import board
import digitalio
import busio

print("Hello, blinka!")

# Try to create a Digital input
pin = digitalio.DigitalInOut(board.D4)
print("Digital IO ok!")

# Try to create an I2C device
i2c = busio.I2C(board.SCL, board.SDA)
print("I2C ok!")

# Try to create an SPI
device spi = busio.SPI(board.SCLK, board.MOSI, board.MISO)
print("SPI ok!")

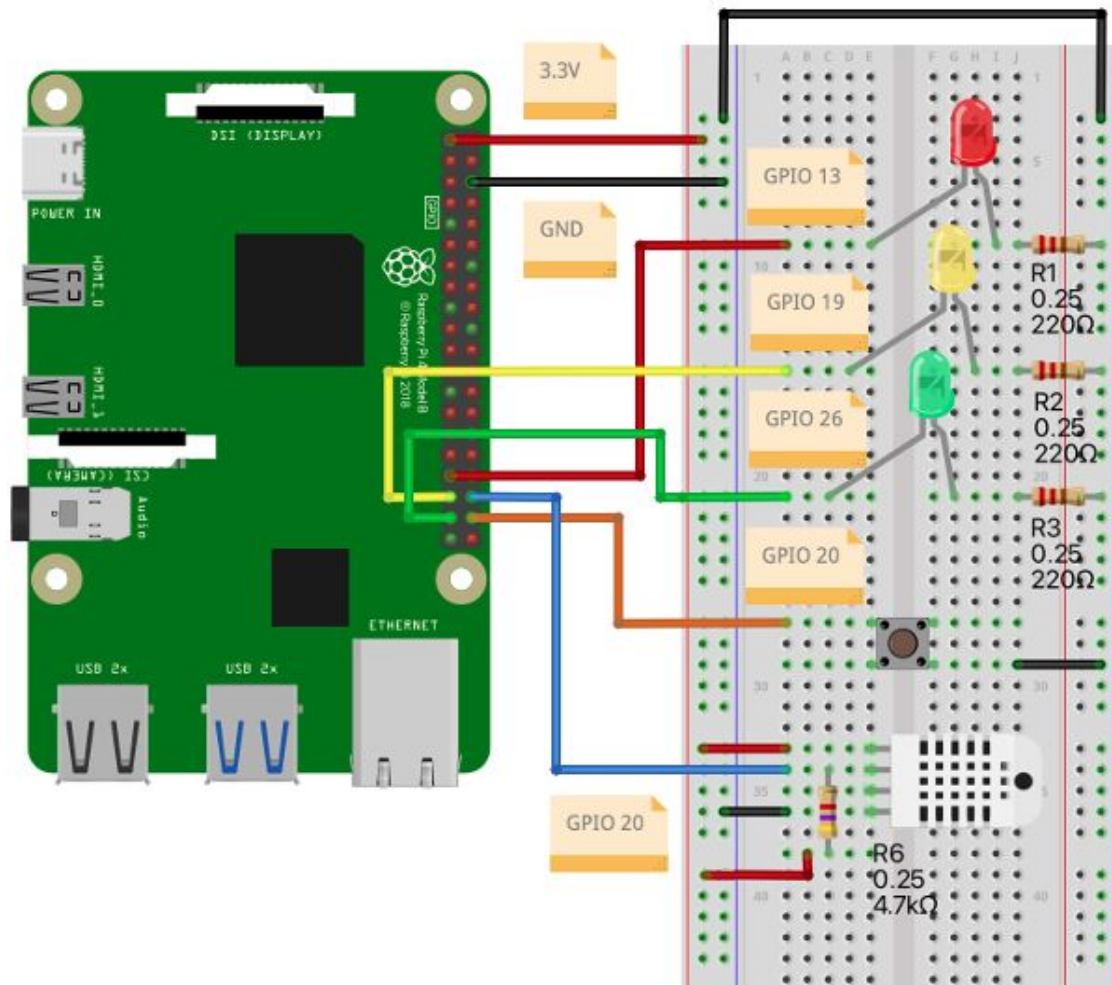
print("done!")
```

blinka_test.py



A terminal window titled 'marcelo_rovai — mjrovai@raspi-5-SD: ~/Documents/GPIO — ssh mjrovai@raspi-5-SD.local — 71x8'. The prompt is '(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO \$'. The user enters 'nano blinka_test.py' and then 'python blinka_test.py'. The output is: 'Hello, blinka!', 'Digital IO ok!', 'I2C ok!', 'SPI ok!', 'done!'. The prompt returns to '(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO \$'.

DHT22 (Temp/Hum Sensor)

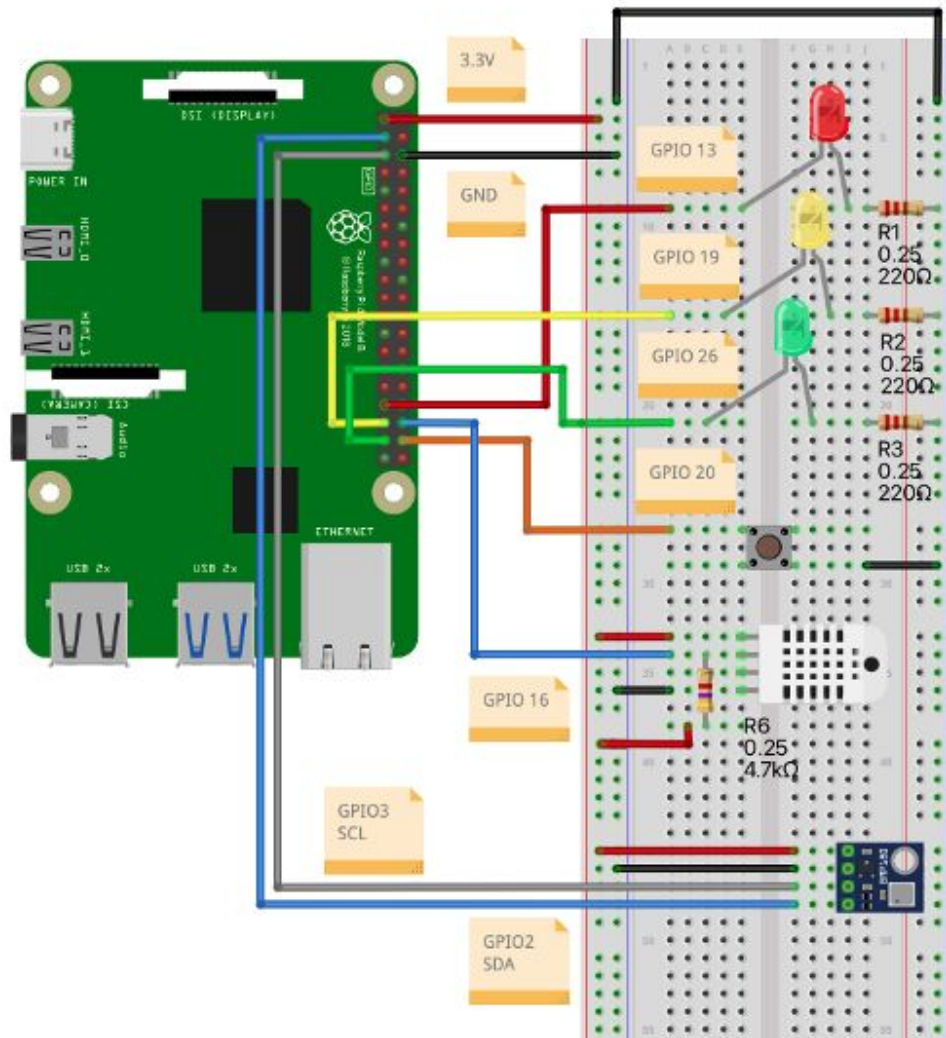


```
pip install adafruit-circuitpython-dht  
python dht_test.py
```

```
marcelo_rovai — mjrovai@raspi-5-SD: ~/Documents/GPIO — ssh mjrovai@raspi-5-SD.local — 83x11  
(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO $ nano dht_test.py  
(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO $ python dht_test.py  
Temp: 72.0 F / 22.2 C      Humidity: 36.8%  
Temp: 70.9 F / 21.6 C      Humidity: 37.2%  
Temp: 70.7 F / 21.5 C      Humidity: 37.2%  
Temp: 70.7 F / 21.5 C      Humidity: 37.2%  
Temp: 70.7 F / 21.5 C      Humidity: 37.2%  
Temp: 70.9 F / 21.6 C      Humidity: 47.5%  
Temp: 71.8 F / 22.1 C      Humidity: 61.4%  
Temp: 73.2 F / 22.9 C      Humidity: 60.3%  
Temp: 74.7 F / 23.7 C      Humidity: 56.0%
```

[dht_test.py](#)

BMP280 (Pressure Sensor)



```
pip install adafruit-circuitpython-bmp280
python bmp280Test.py
```

```
marcelo_rovai — mjrovai@raspi-5-SD: ~/Documents/GPIO — ssh mjrovai@raspi-5-SD.local — 70...
(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO $ nano bmp280_test.py
(ollama) mjrovai@raspi-5-SD:~/Documents/GPIO $ python bmp280_test.py

Temperature: 22.7 C
Pressure: 908.6 hPa
Altitude = 910.07 meters

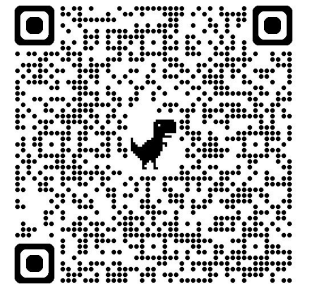
Temperature: 22.7 C
Pressure: 908.6 hPa
Altitude = 909.57 meters
```

[bmp280_test.py](#)



Exploring GPIOs with a Notebook

[Monitoring_Actuating_GPIOs.ipynb](#)



 IMPORTANT: The Notebook Kernel should end after using to liberate GPIOs

The problem is that **Jupyter Notebook is still holding onto the GPIO pins** even after our code finishes running. When we try to run it from the terminal, those pins are already claimed by the Jupyter process. So, before running a code that deals with GPIO in the terminal, In Jupyter:

- Click **Kernel → Restart Kernel**
- Or: Click **Kernel → Shutdown Kernel**

Questions?



Prof. Marcelo J. Rovai

rovai@unifei.edu.br



UNIFEI